Руководство программиста

Описание процесса функционирования ПО:

Пользователь взаимодействует с программой через командную строку. Формат команды:

<название программы> [опции] <имя архива> [файлы, которые необходимо включить в архив]

Программа поддерживает четыре основные операции: создание (c), извлечение (x), добавление (r) и просмотр (t). Опция v (verbose) включает подробный вывод, f указывает имя архива.

Типовые сценарии:

- Создание нового архива

  Цель**:** упаковать несколько файлов в один архивный файл.

  Действия пользователя(здесь и далее, программа будет вызываться по имени exp):

  «./exp cf backup.tar document.txt photo.jpg»

Что происходит в системе:

1. Программа открывает файл backup.tar для записи в бинарном режиме.
2. Для каждого переданного файла формируется заголовок (512 байт) с метаданными: путь, права доступа, UID, GID, размер, время модификации.
3. Вычисляется контрольная сумма заголовка и записывается в поле chksum.
4. Заголовок и содержимое файла (с выравниванием до 512 байт) записываются в архив.
5. В конце добавляются два пустых блока (1024 байта нулей) – признак конца архива.
6. При включённой опции v в консоль выводится имя каждого добавленного файла.

   Результат: создан файл backup.tar, содержащий все указанные файлы.

- Просмотр содержимого архива

  Цель**:** узнать, какие файлы находятся в архиве, не извлекая их.

  Действия пользователя:

  ./exp tf backup.tar

  Или в случае необходимости подробной информации:

  ./exp tfv backup.tar

Что происходит в системе:

1. Программа открывает архив для чтения.
2. Последовательно считывает заголовки записей, пропуская пустые блоки.
3. Для каждого найденного файла проверяется контрольная сумма заголовка. Если она не совпадает – выводится предупреждение, но чтение продолжается (пропускаются блоки данных).
4. При обычном режиме (t) выводится только имя файла. При -tv дополнительно выводятся права доступа (восьмеричное), размер и время модификации.
5. После вывода программа завершается.

   Результат: пользователь видит список файлов в архиве.

- Извлечение файлов из архива

  Цель**:** восстановить файлы из архива в текущую директорию.

  Действия пользователя:

  ./exp xf backup.tar

  Или в случае необходимости извлечь конкретные файлы:

  ./exp xf backup.tar document.txt

Что происходит в системе:

1. Архив открывается для чтения.
2. Программа перебирает записи, сравнивая имена файлов с переданными (если они указаны). Если список не задан – извлекаются все.

3. Для каждого выбранного файла проверяется контрольная сумма заголовка. Если она не совпадает – выводится ошибка и файл пропускается.
4. Считывается содержимое файла (учитывая выравнивание блоков) и записывается в файловую систему с восстановлением прав доступа и времени модификации (через chmod и utime).
5. При включённой -v выводится имя каждого извлечённого файла.

   Результат: в текущем каталоге появляются извлечённые файлы с исходными атрибутами.

- Добавление файлов в существующих архив

  Цель**:** дополнить уже созданный архив новыми файлами без перезаписи всего архива.

  Действия пользователя:

  ./exp rf backup.tar newfile.txt

Что происходит в системе:

1. Архив открывается для чтения и записи (режим обновления).
2. Указатель устанавливается на конец архива (перед финальными нулевыми блоками).
3. Заново вызывается процедура создания записей для новых файлов.
4. В конце дописываются два нулевых блока (старые конечные блоки перезаписываются новыми).

   Результат: Архив будет дополнен новыми файлами.

После выполнения операции программа выводит приглашение (ничего, если нет ошибок) и завершается с кодом 0. При возникновении критических ошибок (невозможность открыть архив, неисправность файлов) выводится диагностическое сообщение и код возврата 1.

Описание процесса информационного взаимодействия:

Входные потоки данных:

- Поток ввода консоли(всегда);

- Поток чтения архива (если требуется извлечение или узнать содержимое архива);
- Поток чтения файлов (если архив создаётся или дополняется).

Выходные потоки данных:

- Поток вывода консоли (при запросе данных об архиве и/или обработанных файлах);
- Поток записи архива (для дозаписи или при создании);
- Поток записи файлов (при извлечении).

Каждый файловый объект включает любые данные файла и предваряется 512-байтовой записью заголовка. Данные файла записываются без изменений, за исключением того, что его длина округляется до кратного 512 байтам. Заголовок содержит в себе ряд метаданых файла, необходимых для сохранения достаточной информации для дальнейшего воссоздания этого файла из архива.

Обработка ошибок и нештатные ситуации:

- Указан несуществующий архив для извлечения: программа выдаёт сообщение об ошибке открытия файла и завершается с кодом возврата 1.
- Повреждённый архив (неверная контрольная сумма): при извлечении или просмотре выводится предупреждение file is broken: имя_файла, и программа пропускает этот файл (перемещается к следующей записи), чтобы не прерывать работу.
- Попытка добавить файл в архив, который не существует: при использовании r без существующего архива программа создаст новый архив.
- Превышение лимита длины имени файла (>100 символов): имя обрезается до 100, что может привести к коллизиям имён.

Описание состава ПО:

Заголовочный файл header.h:

Назначение – определение формата записи архива.

Содержит:

- Константу RECORDSIZE (512 байт) – размер блока данных в архиве.
- Константу NAMSIZE (100) – максимальная длина имени файла.
- Объединение union record – представляет один 512-байтовый блок. Структура header содержит:
    - path[100] – имя файла,
    - mode[8] – права доступа,
    - uid[8], gid[8] – идентификаторы пользователя и группы,
    - size[12] – размер файла в байтах,
    - mtime[12] – время последней,
    - chksum[8] – поле для контрольной суммы заголовка,
    - linkflag – флаг типа файла,
    - arch_linkname[100] – имя ссылки

Файл event.h – класс event:

Назначение – управление параметрами командной строки, состоянием опций и файловым потоком архива.

Перечисление parameters – определяет возможные режимы работы:

- none – режим не установлен,
- create – создание архива,
- append – добавление файлов в существующий архив,
- extract – извлечение файлов,
- list – вывод списка содержимого.

Члены класса:

- int argc – количество аргументов командной строки,
- std::vector<std::string> arguments – вектор аргументов,
- std::fstream file – поток для работы с файлом архива,
- parameters options – выбранный режим,
- bool verbose – логический индикатор подробного вывода,
- bool addEmptyBlocks – флаг добавления двух пустых блоков в конец архива (устанавливается для режима создания).

Ключевые методы класса:

- Конструктор event(int p_argc, char** p_argv) – сохраняет количество аргументов и копирует их в вектор arguments, инициализирует опции значениями по умолчанию.
- void Parser() – анализирует аргументы командной строки:
    1. Проверяет минимальное количество аргументов (не менее 3).
    2. Ищет в arguments[1] ключевые символы: 'c' (create), 'r' (append), 'x' (extract), 't' (list).
    3. Проверяет некорректные комбинации.
    4. Вызывает соответствующий приватный метод для настройки опций и открытия файла.
    5. При наличии опции v устанавливает verbose=true.
    6. Если присутствует f, выводит имя файла архива (из arguments[2]).

Файл main.cpp – реализация операций архиватора

Назначение – содержит функции для создания, добавления, извлечения и просмотра архива, а также точку входа main.

Вспомогательные и основные функции:

unsigned int calcChecksum(union record *r):

- Вычисляет контрольную сумму для заголовка записи.
- Обнуляет поле chksum, суммирует все 512 байт блока и записывает результат в восьмеричном формате в поле chksum. Возвращает вычисленную сумму.

void Creation(event& Event):

- Реализует создание нового архива (опция c).
- Для каждого файла, переданного в аргументах:
    - Заполняет заголовок
    - Вычисляет контрольную сумму и записывает 512-байтовый блок в архив.
    - Если файл является обычным и его размер > 0, читает его содержимое блоками по 4096 байт и записывает в архив, дополняя последний блок нулями до кратности 512.
- При включённом verbose выводит имена обработанных файлов.
- Если установлен флаг addEmptyBlocks, дописывает два нулевых блока – маркер конца архива.

static void Append(event& Event):

- Добавляет файлы в конец существующего архива с отступом от конца в два блока по 512 байт.
- После этого снова дописывает два нулевых блока, чтобы сохранить корректную структуру архива.

static void Extract(event& Event):

- Извлекает файлы из архива.

- Последовательно читает 512-байтовые блоки:

Если файл подлежит извлечению:

1. Создаёт выходной файл с тем же именем.
2. Читает fileSize байт данных из архива (с учётом выравнивания) и записывает в файл.
3. После извлечения восстанавливает права доступа (chmod) и время модификации (utime) согласно заголовку.
4. При verbose выводит имя файла.

Если файл не извлекается, просто пропускает его данные (перемещает указатель на размер, кратный 512).

void List(event& Event):

- Выводит список содержимого архива.
- Читает записи аналогично Extract, пропуская пустые блоки.
- Проверяет контрольную сумму и в случае ошибки выводит предупреждение, но продолжает обход.
- Для каждого файла, если он подлежит отображению (все или перечисленные в аргументах), выводит:
    1. только имя, если verbose выключен;
    2. права доступа (в восьмеричном виде), размер, время модификации и имя, если verbose включён.
- Пропускает данные файла, чтобы перейти к следующей записи.

int main(int argc, char* argv[]):

- Точка входа.
- Создаёт объект event CoreEvent, передавая аргументы командной строки.

- Вызывает CoreEvent.Parser() для разбора параметров и открытия файла архива.
- В зависимости от полученного режима (getOption()) вызывает соответствующую функцию:
    1. parameters::create → Creation(CoreEvent),
    2. parameters::append → Append(CoreEvent),
    3. parameters::extract → Extract(CoreEvent),
    4. parameters::list → List(CoreEvent),
    5. default – ничего не делает.

Взаимодействие компонентов:

- header.h предоставляет структуру данных для всех операций с записями архива.
- event.h инкапсулирует логику разбора командной строки и предоставляет унифицированный доступ к параметрам и файловому потоку для остальных функций.
- main.cpp реализует всю бизнес-логику архиватора, используя объект event и структуры из header.h. Функции в этом файле непосредственно работают с файловым потоком, читают/записывают заголовки и данные файлов.